

第4章 字符串与正则表达式

《Python程序设计，董付国》

4.1 字符串

- 最早的字符串编码是美国标准信息交换码ASCII，仅对10个数字、26个大写英文字母、26个小写英文字母及一些其他标点符号和控制符号进行了编码。ASCII码采用1个字节来对字符进行编码，最多只能表示256个符号。

4.1 字符串

- 随着信息技术的发展和信息交换的需要，各国的文字都需要进行编码，不同的应用领域和场合对字符串编码的要求也略有不同，于是又分别设计了多种不同的编码格式，常见的主要有UTF-8、UTF-16、UTF-32、GB2312、GBK、CP936等。

4.1 字符串

- **GB2312**是我国制定的中文编码，使用1个字节兼容ASCII，2个字节表示中文；GBK是GB2312的扩充，而CP936是微软在GBK基础上开发的编码方式。**GB2312、GBK和CP936互相兼容，都是使用2个字节表示中文。**
- **UTF-8**对全世界所有国家需要用到的字符进行了编码，以1个字节表示英语字符（兼容ASCII），以**3个字节表示常见汉字**，还有些语言的符号使用2个字节（例如俄语和希腊语符号）或者4个字节。
- **UTF-8**字符集比**GBK**字符集大很多，有些字符没有对应的**GBK**编码。

4.1 字符串

- 不同编码格式之间相差很大，采用不同的编码格式意味着不同的表示和存储形式，把同一字符存入不同编码格式的文件时，写入的内容可能会不同，在试图理解文件内容时必须了解编码规则并进行正确的解码。如果解码方法不正确就无法还原信息，从这个角度来讲，字符串编码也具有加密的效果。

4.1 字符串

```
>>> '董付国'.encode('utf8')
b'\xe8\x91\xa3\xe4\xbb\x98\xe5\x9b\xbd'
>>> '董付国'.encode('cp936')
b'\xb6\xad\xb8\xb6\xb9\xfa'
>>> '董付国'.encode('cp936').decode('cp936')
'董付国'
>>> 'Python可以这样学'.encode('utf8').decode('cp936')
# 解码格式与编码格式不兼容，失败
UnicodeDecodeError: 'gbk' codec can't decode byte 0xaf in position 8:
illegal multibyte sequence
>>> 'Python程序设计开发宝典'.encode('cp936').decode('utf8')
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xb3 in position 6:
invalid start byte
>>> '测试'.encode('utf8').decode('gbk') # 格式不兼容，无法还原内容
'嫵嫵嫵'
```

4.1 字符串

- Python 3.x完全支持中文字符，Python程序文件默认使用UTF8编码格式，无论是一个数字、英文字母，还是一个汉字，在统计字符串长度时都按一个字符对待和处理。

```
>>> s = '中国山东烟台'
```

```
>>> len(s)                                # 字符串长度，或者包含的字符个数
```

```
6
```

```
>>> s = 'Python小屋'
```

```
# 中文与英文字符同样对待，都算一个字符
```

```
>>> len(s)
```

```
8
```

```
>>> 姓名 = '张三'
```

```
# 可以使用中文作为变量名，但不能包含中文标点符号
```

```
>>> print(姓名)
```

```
# 输出变量的值
```

```
张三
```

4.1.1 字符串格式化

■ 常用格式字符

格式字符	说明
%s	字符串（采用str()的显示）
%r	字符串（采用repr()的显示）
%c	单个字符
%d	十进制整数
%i	十进制整数
%o	八进制整数
%x	十六进制整数
%e	指数（基底写为e）
%E	指数（基底写为E）
%f、%F	浮点数
%g	指数(e)或浮点数（根据显示长度确定）
%G	指数(E)或浮点数（根据显示长度确定）
%%	一个字符%

4.1.1 字符串格式化

```
>>> x = 1235
```

```
>>> '%o' % x
```

```
'2323'
```

```
>>> '%#x' % x
```

使用#时, 它会在前面加上0x前缀

```
'0x4d3'
```

```
>>> '%e' % x
```

```
'1.235000e+03'
```

```
>>> '%s' % 65
```

```
'65'
```

```
>>> '%s' % 65333
```

```
'65333'
```

```
>>> '%s' % [1, 2, 3]
```

直接把对象转换成字符串

```
'[1, 2, 3]'
```

```
>>> '%d' % '555'
```

无法按整数格式对字符串进行格式化

```
TypeError: %d format: a number is required, not str
```

4.1.1 字符串格式化

■ 使用字符串方法format()进行格式化

```
>>> print('The number {0:}, in hex is: {0:#x}, the number {1} in oct is  
      {1:#o}'.format(5555, 55))
```

```
The number 5,555 in hex is: 0x15b3, the number 55 in oct is 0o67
```

```
>>> print('The number {1:}, in hex is: {1:#x}, the number {0} in oct is  
      {0:o}'.format(5555, 55))      # 注意格式o和#o的区别 (0x), :,表示千位分隔符, 如55,000
```

```
The number 55 in hex is: 0x37, the number 5555 in oct is 12663
```

```
>>> print('my name is {name}, my age is {age}, and my QQ is {qq}'.format(name='Dong  
      Fuguo', age=40, qq='30646****'))
```

```
my name is Dong Fuguo, my age is 40, and my QQ is 30646****
```

```
>>> position = (5, 8, 13)
```

```
>>> print('X:{0[0]};Y:{0[1]};Z:{0[2]}'.format(position))
```

```
X:5;Y:8;Z:13
```

```
>>> '{0:<8d},{0:^8d},{0:>8d}'.format(65)  # 设置对齐方式
```

```
'65      ,      65      ,      65'
```

```
>>> '{0:+<8d},{0:-^8d},{0:=>8d}'.format(65)
```

```
'65+++++,---65---,=====65'
```

4.1.1 字符串格式化

```
>>> print('{0:.4f}'.format(10/3))          # 指定小数位数
3.3333
>>> print('{0:.2%}'.format(1/3))          # 格式化为百分数
33.33%
>>> print('{0:>10.2%}'.format(1/3))       # 指定宽度和对齐方式
      33.33%
>>> print('{:c},{:d},{:e}'.format(65, 66, 67))
A,66,6.700000e+01
# 直接按顺序对多个参数格式化
>>> print('{0:_},{0:#_x}'.format(10000000))
10_000_000,0x98_9680
# 使用下划线做千分符
```

4.1.1 字符串格式化

```
weather = [("Monday", "rainy"), ("Tuesday", "sunny"),  
           ("Wednesday", "sunny"), ("Thursday", "rainy"),  
           ("Friday", "cloudy")]
```

```
formatter = "Weather of '{0[0]}' is '{0[1]}'.format
```

```
for item in map(formatter, weather):
```

```
    print(item)
```

```
for item in weather:
```

```
    print(formatter(item))
```

等价

运行结果:

Weather of 'Monday' is 'rainy'

Weather of 'Tuesday' is 'sunny'

Weather of 'Wednesday' is 'sunny'

Weather of 'Thursday' is 'rainy'

Weather of 'Friday' is 'cloudy'

4.1.2 字符串常用方法

■ find()、rfind()、index()、rindex()、count()

✓ find()和rfind方法分别用来查找另一个字符串在当前字符串指定范围（默认是整个字符串）中首次和最后一次出现的位置，如果不存在则返回-1；

✓ index()和rindex()方法用来返回另一个字符串在当前字符串指定范围中首次和最后一次出现的位置，如果不存在则抛出异常；

✓ count()方法用来返回另一个字符串在当前字符串中出现的次数，不存在时返回0.

4.1.2 字符串常用方法

```
>>> s = "apple,peach,banana,peach,pear"
>>> s.find("peach")
6
>>> s.find("peach", 7)
19
>>> s.find("peach", 7, 20)
-1
>>> s.rfind('p')
25
>>> s.index('p')
1
>>> s.index('pe')
6
```

```
>>> s.index('pear')
25
>>> s.index('ppp')
Traceback (most recent call
last):
  File "<pyshell#11>", line 1,
    in <module>
      s.index('ppp')
ValueError: substring not found
>>> s.count('p')
5
>>> s.count('pp')
1
>>> s.count('ppp')
0
```

4.1.2 字符串常用方法

- `split()`、`rsplit()`、`partition()`、`rpartition()`
- ✓ `split()`和`rsplit()`方法分别用来以指定字符为分隔符，把当前字符串从左往右或从右往左分隔成多个字符串，并返回包含分隔结果的列表；
- ✓ `partition()`和`rpartition()`用来以指定字符串为分隔符将原字符串分隔为3部分，分别为分隔符前的字符串、分隔符字符串、分隔符后的字符串，如果指定的分隔符不在原字符串中，则返回原字符串和两个空字符串。

4.1.2 字符串常用方法

- 对于split()和rsplit()方法, 如果**不指定分隔符**, 则字符串中的任何空白符号(空格、换行符、制表符等)都将被认为是分隔符, 并**删除切分结果中的空字符串**。

```
>>> s = 'hello world \n\n My name is Dong '
>>> s.split()
['hello', 'world', 'My', 'name', 'is', 'Dong']
>>> s = '\n\nhello world \n\n\n My name is Dong '
>>> s.split()
['hello', 'world', 'My', 'name', 'is', 'Dong']
>>> s = '\n\nhello\t\t world \n\n\n My name\t is Dong '
>>> s.split()
['hello', 'world', 'My', 'name', 'is', 'Dong']
```

4.1.2 字符串常用方法

■ 字符串连接join()

```
>>> li = ['apple', 'peach', 'banana', 'pear']
>>> ','.join(li)           # 使用逗号作为连接符
'apple,peach,banana,pear'
>>> '.'.join(li)
'apple.peach.banana.pear'
>>> '::'.join(li)
'apple::peach::banana::pear'
```

4.1.2 字符串常用方法

■ 不推荐使用+运算符连接字符串，优先使用join()方法。

```
import timeit
```

```
# 使用列表推导式生成10000个字符串
```

```
strlist = ['This is a long string that will not keep in memory.'  
           for n in range(10000)]
```

```
# 使用字符串对象的join()方法连接多个字符串
```

```
def use_join():  
    return ''.join(strlist)
```

```
# 使用运算符+连接多个字符串
```

```
def use_plus():  
    result = ''  
    for strtemp in strlist:  
        result = result+strtemp  
    return result
```

4.1.2 字符串常用方法

- `timeit`模块还支持下面代码演示的用法，从运行结果可以看出，当需要对大量数据进行类型转换时，内置函数`map()`可以提供非常高的效率。

```
>>> import timeit
>>> timeit.timeit('"-".join(str(n) for n in range(100))', number=10000)
# 运行10000次
0.3063435900577929
>>> timeit.timeit('"-".join([str(n) for n in range(100)])', number=10000)
0.27191914957273866
>>> timeit.timeit('"-".join(map(str, range(100)))', number=10000)
0.21119518171659024
```

4.1.2 字符串常用方法

■ lower()、upper()、capitalize()、title()、swapcase()

```
>>> s = "What is Your Name?"
```

```
>>> s.lower() # 返回小写字符串
```

```
'what is your name?'
```

```
>>> s.upper() # 返回大写字符串
```

```
'WHAT IS YOUR NAME?'
```

```
>>> s.capitalize() # 字符串首字符大写
```

```
'What is your name?'
```

```
>>> s.title() # 每个单词的首字母大写
```

```
'What Is Your Name?'
```

```
>>> s.swapcase() # 大小写互换
```

```
'wHAT IS yOUR nAME?'
```

4.1.2 字符串常用方法

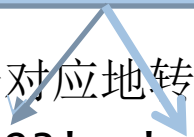
- `replace()` 方法，类似于Word中的“全部替换”功能，返回处理后的新字符串，不修改原来的字符串内容。

```
>>> words = ('测试', '非法', '暴力', '话')
>>> text = '这句话里含有非法内容'
>>> for word in words:
    if word in text:
        text = text.replace(word, '***')
>>> text
'这句***里含有***内容'
```

4.1.2 字符串常用方法

- 字符串对象的`maketrans()`方法用来生成字符映射表，而`translate()`方法用来根据映射表中定义的对应关系转换字符串并替换其中的字符，使用这两个方法的组合可以同时处理多个字符。

这两个参数不是作为
整体进行处理的



创建映射表，将字符"abcdef123"一一对应地转换为"uvwxyz@#\$\$"

```
>>> table = ''.maketrans('abcdef123', 'uvwxyz@#$$')
```

```
>>> s = "Python is a great programming language. I like it!"
```

```
>>> s.translate(table) # 按映射表进行替换
```

```
'Python is u gryut progrumming lunguugy. I liky it!'
```

4.1.2 字符串常用方法

```
>>> table = ''.maketrans('0123456789', '零一二三四伍陆柒捌玖')  
>>> '2023年12月31日'.translate(table)  
'二零二三年一二月三一日'
```

4.1.2 字符串常用方法

- 应用：凯撒加密，每个字母替换为后面第k个。

```
from string import ascii_letters, ascii_lowercase, ascii_uppercase
```

```
def kaisaEncrypt(text, k):  
    # 凯撒加密  
    lower = ascii_lowercase[k:] + ascii_lowercase[:k]  
    upper = ascii_uppercase[k:] + ascii_uppercase[:k]  
    table = ''.maketrans(ascii_letters, lower+upper)  
    return text.translate(table)
```

```
s = "Python is a great programming language. I like it!"  
print(kaisaEncrypt(s, 3))
```

```
'Sbwkrq lv d juhdw surjudpplqj odqjxdjh. L olnh lw!'
```

4.1.2 字符串常用方法

- 应用：凯撒解密。

```
def check(text):  
    # 测试文本中是否存在至少两个最常见的英语单词  
    # 列表可扩展，规则可调整  
    mostCommonWords = ('the', 'is', 'to', 'not', 'have', 'than', 'for')  
    return sum(1 for word in mostCommonWords if word in text)>=2  
  
# 测试  
text = 'Beautiful is better than ugly.'  
# 加密  
encryptedText = kaisaEncrypt(text, 5)  
# 暴力破解  
for i in range(1, 26):  
    t = kaisaEncrypt(encryptedText, -i)  
    if check(t):  
        print(i, t, sep=':')  
        break
```

4.1.2 字符串常用方法

- 这三个方法的参数指定的字符串并不作为一个整体对待，而是在原字符串的两侧、右侧、左侧删除参数字符串中包含的所有字符，一层一层地从外往里扒。

>>> 'aabbccddeeffg'.strip('af') # 字母f不在字符串两侧，所以不删除

'bbccddeeffg'

>>> 'aabbccddeeffg'.strip('gaf')

'bbccddeee'

>>> 'aabbccddeeffg'.strip('gaef')

'bbccdd'

>>> 'aabbccddeeffg'.strip('gbaef')

'ccdd'

>>> 'aabbccddeeffg'.strip('gbaefcd')

''

4.1.2 字符串常用方法

- 应用：文本规范化。

```
>>> text = '''姓名：张三
```

```
年龄：39
```

```
性别男
```

```
职业  学生
```

```
籍贯：  地球'''
```

```
>>> infomation = text.split('\n')
```

```
>>> infomation
```

```
['姓名：张三', '年龄：39', '性别男', '职业  学生', '籍贯：  地球']
```

```
>>> for item in infomation:
```

```
    print(item[:2], item[2:].strip(': '), sep=': ')
```

```
姓名：张三
```

```
年龄：39
```

```
性别：男
```

```
职业：学生
```

```
籍贯：地球
```

4.1.2 字符串常用方法

■ 内置函数eval()

```
>>> eval("3+4")
```

```
7
```

```
>>> a = 3
```

```
>>> b = 5
```

```
>>> eval('a+b')
```

```
8
```

```
>>> import math
```

```
>>> eval('math.sqrt(3)')
```

```
1.7320508075688772
```

```
>>> eval('aa')
```

```
NameError: name 'aa' is not defined
```

```
>>> eval('*'.join(map(str, range(1, 6))))
```

```
120
```

计算表达式的值

要求变量a和b已存在

当前上下文中不存在对象aa

5的阶乘

4.1.2 字符串常用方法

❖ eval() 函数是非常危险的

```
>>> a = input('Please input:')
```

```
Please input: __import__('os').startfile(r'C:\Windows\notepad.exe')
```

```
>>> eval(a)
```

```
>>> eval("__import__('os').system('md testtest')")
```

4.1.2 字符串常用方法

■ 关键字in

```
>>> 'a' in 'abcde'      # 测试一个字符中是否存在于另一个字符串中
True
>>> 'ab' in 'abcde'
True
>>> 'ac' in 'abcde'      # 关键字in左边的字符串作为一个整体对待
False
>>> 'j' in 'abcde'
False
```

4.1.2 字符串常用方法

- Python字符串支持与**整数**的乘法运算，表示序列重复，也就是**字符串内容的重复**，得到新字符串。

```
>>> 'abcd' * 3  
'abcdabcdabcd'
```

4.1.2 字符串常用方法

■ `s.startswith(t)`、`s.endswith(t)`，判断字符串是否以指定字符串开始或结束

```
>>> s = 'Beautiful is better than ugly.'
```

```
>>> s.startswith('Be')           # 检测整个字符串
```

```
True
```

```
>>> s.startswith('Be', 5)        # 指定检测范围起始位置
```

```
False
```

```
>>> s.startswith('Be', 0, 5)     # 指定检测范围起始和结束位置
```

```
True
```

```
>>> import os
```

```
>>> [filename for filename in os.listdir(r'c:\\')  
     if filename.endswith(('.bmp', '.jpg', '.gif'))]
```

4.1.2 字符串常用方法

- `isalnum()`、`isalpha()`、`isdigit()`、`isdecimal()`、`isnumeric()`、`isspace()`、`isupper()`、`islower()`，用来测试字符串是否全部为数字或英文字母、是否全部为英文字母、是否全部为数字字符、是否全部为空白字符、英文字母是否全部为大写以及英文字母是否全部为小写。

```
>>> '1234abcd'.isalnum()
```

```
True
```

```
>>> '1234abcd'.isalpha()
```

```
# 全部为英文字母时返回True
```

```
False
```

```
>>> '1234abcd'.isdigit()
```

```
# 全部为数字时返回True
```

```
False
```

```
>>> 'abcd'.isalpha()
```

```
True
```

```
>>> '1234.0'.isdigit()
```

```
False
```

4.1.2 字符串常用方法

```
>>> '1234'.isdigit()
```

```
True
```

```
>>> '九'.isnumeric()
```

isnumeric()方法支持汉字数字

```
True
```

```
>>> '九'.isdigit()
```

```
False
```

```
>>> '九'.isdecimal()
```

```
False
```

```
>>> 'IVIII X'.isdecimal()
```

```
False
```

```
>>> 'IVIII X'.isdigit()
```

```
False
```

```
>>> 'IVIII X'.isnumeric()
```

支持罗马数字

```
True
```

4.1.2 字符串常用方法

- 除了字符串对象提供的方法以外，很多Python内置函数也可以对字符串进行操作，例如：

```
>>> x = 'Hello world.'
>>> max(x), min(x), len(x)
('w', ' ', 12)
>>> max(['abc', 'ABD'], key=str.upper) # 忽略大小写
'ABD'
>>> sorted(x)
[' ', '.', 'H', 'd', 'e', 'l', 'l', 'l', 'o', 'o', 'r', 'w']
>>> list(zip(x,x)) # zip()也可以作用于字符串
[('H', 'H'), ('e', 'e'), ('l', 'l'), ('l', 'l'), ('o', 'o'), (' ', ' '),
('w', 'w'), ('o', 'o'), ('r', 'r'), ('l', 'l'), ('d', 'd'), ('.', '.')]
>>> eval('[1, 2, 3, 4]') # 字符串求值
[1, 2, 3, 4]
```

4.1.2 字符串常用方法

■ 切片也适用于字符串，但仅限于读取其中的元素，不支持字符串修改。

```
>>> 'Explicit is better than implicit.'[:8]
```

```
'Explicit'
```

```
>>> 'Explicit is better than implicit.'[9:23]
```

```
'is better than'
```

4.1.4 可变字符串

- 在Python中，字符串属于不可变对象，不支持原地修改，如果需要修改其中的值，只能重新创建一个新的字符串对象。然而，如果确实需要一个支持原地修改的unicode数据对象，可以使用io.StringIO对象或array模块。

```
>>> import io
>>> s = "Hello, world"
>>> sio = io.StringIO(s)
>>> sio.getvalue()
'Hello, world'
>>> sio.seek(7)
7
>>> sio.write("there!")
6
>>> sio.getvalue()
'Hello, there!'
```

```
>>> import array
>>> a = array.array('u', s)
>>> print(a)
array('u', 'Hello, world')
>>> a[0] = 'y'
>>> print(a)
array('u', 'yello, world')
>>> a.tounicode()
'yello, world'
```

4.1.5 中文分词与拼音处理

```
>>> import jieba                                # 导入jieba模块
>>> x = '分词的准确度直接影响了后续文本处理和挖掘算法的最终效果。'
>>> jieba.cut(x)                                # 使用默认词库进行分词
<generator object Tokenizer.cut at 0x00000000342C990>
>>> list(_)
['分词', '的', '准确度', '直接', '影响', '了', '后续', '文本处理', '和', '挖
掘', '算法', '的', '最终', '效果', '。']
>>> jieba.lcut('Python可以这样学，Python程序设计开发宝典') # 直接给出列表
Dumping model to file cache C:\Users\d\AppData\Local\Temp\jieba.cache
['Python', '可以', '这样', '学', '，', '，', 'Python', '程序设计', '开发', '宝典']
>>> list(jieba.cut('花纸杯'))
['花', '纸杯']
>>> jieba.add_word('花纸杯')                    # 增加词条
>>> list(jieba.cut('花纸杯'))                  # 使用新词库进行分词
['花纸杯']
```

4.1.5 中文分词与拼音处理

```
>>> from jieba import posseg
>>> text = '分词的准确度直接影响了后续文本处理和挖掘算法的最终效果。'
>>> for word, tag in posseg.cut(text): # 得到分词及其词性
    print(word, tag, sep=':', end='\t')
```

分词:n 的:uj 准确度:n 直接:ad 影响:vn 了:u1 后续:v
文本处理:n 和:c 挖掘:v 算法:n 的:uj 最终:d 效
果:n 。:x

4.1.5 中文分词与拼音处理

```
>>> import snownlp                                # 导入snownlp模块
>>> snownlp.SnowNLP('学而时习之，不亦说乎').words
['学而', '时习', '之', '，', '，', '不亦', '说乎']
>>> snownlp.SnowNLP(x).words
['分词', '的', '准确度', '直接', '影响', '了', '后续', '文本', '处理', '和', '挖掘', '算法', '的', '最终', '效果', '。']
```

4.1.5 中文分词与拼音处理

- 应用：过滤字符串中的空白字符和中英文标点符号。

```
from jieba import cut
```

```
def delPuncs(s):
```

```
    f = lambda word: len(word)>1 or '\u4e00'<=word<='\u9fa5'
```

```
    return ''.join(filter(f, cut(s)))
```

```
sentence = '''
```

东边来个小朋友叫小松，手里拿着一捆葱。西边来个小朋友叫小丛，手里拿着小闹钟。

小松手里葱捆得松，掉在地上一些葱。小丛忙放闹钟去拾葱，帮助小松捆紧葱。

小松夸小丛像雷锋，小丛说小松爱劳动。'''

```
print(delPuncs(sentence))
```

4.1.5 中文分词与拼音处理

```
>>> from pypinyin import lazy_pinyin, pinyin
>>> lazy_pinyin('董付国')           # 返回拼音
['dong', 'fu', 'guo']
>>> lazy_pinyin('董付国', 1)        # 带声调的拼音
['dǒng', 'fù', 'guó']
>>> lazy_pinyin('董付国', 2)        # 另一种拼音形式，数字表示前面字母的声调
['do3ng', 'fu4', 'guo2']
>>> lazy_pinyin('董付国', 3)        # 只返回拼音的声母
['d', 'f', 'g']
>>> lazy_pinyin('重要', 1)          # 能够根据词组智能识别多音字
['zhòng', 'yào']
>>> lazy_pinyin('重阳', 1)
['chóng', 'yáng']
>>> pinyin('重阳')                 # 返回拼音
[['chóng'], ['yáng']]
>>> pinyin('重阳节', heteronym=True) # 返回多音字的所有读音
[['chóng'], ['yáng'], ['jié', 'jiē']]
```

4.1.5 中文分词与拼音处理

```
>>> import jieba                # 其实不需要导入jieba, 这里只是说明已安装
>>> x = '中英文混合test123'
>>> lazy_pinyin(x)              # 自动调用已安装的jieba扩展库分词功能
['zhong', 'ying', 'wen', 'hun', 'he', 'test123']
>>> lazy_pinyin(jieba.cut(x))
['zhong', 'ying', 'wen', 'hun', 'he', 'test123']
>>> x = '山东烟台的大樱桃真好吃啊'
>>> sorted(x, key=lazy_pinyin) # 按拼音对汉字进行排序
['啊', '吃', '大', '的', '东', '好', '山', '台', '桃', '烟', '樱', '真']
>>> ''.join(map(lambda word: ''.join(sorted(word, key=pinyin)) if
len(word)==2 else word, jieba.cut(x)))
'东山台烟的大樱桃真吃好啊'
```

4.1.5 字符串应用案例精选

■ **例4-3** 检查并判断密码字符串的安全强度。

```
import string

def check(pwd):
    # 密码必须至少包含6个字符
    if not isinstance(pwd, str) or len(pwd)<6:
        return 'not suitable for password'
    # 密码强度等级与包含字符种类的对应关系
    d = {1:'weak', 2:'below middle', 3:'above middle', 4:'strong'}
    # 分别用来标记pwd是否含有数字、小写字母、大写字母和指定的标点符号
    r = [False] * 4
```

4.1.5 字符串应用案例精选

```
for ch in pwd:
    # 检查当前字符是否为数字
    if not r[0] and ch in string.digits:
        r[0] = True
    # 检查当前字符是否为小写字母
    elif not r[1] and ch in string.ascii_lowercase:
        r[1] = True
    # 检查当前字符是否为大写字母
    elif not r[2] and ch in string.ascii_uppercase:
        r[2] = True
    # 检查当前字符是否为指定的标点符号
    elif not r[3] and ch in ',.!?;<>':
        r[3] = True
# 统计包含的字符种类，返回密码强度
return d.get(r.count(True), 'error')
```

```
print(check('a2Cd,'))
```

4.1.5 字符串应用案例精选

- 补充：使用集合实现。

```
from string import ascii_lowercase, ascii_uppercase, digits

possible = [set(ascii_lowercase), set(ascii_uppercase), set(digits), set('.,_')]
# 如果密码字符串包含小写字母、大写字母、数字、标点符号中的4种，强密码
# 包含3种表示中高强度，2种表示中低强度，1种为弱密码
security = {1:'weak', 2:'below middle', 3:'above middle', 4:'strong'}

def checkPwd(pwd):
    pwd = set(pwd)
    # 检查密码字符串集合与小写字母、大写字母、
    # 数字字符、标点符号等集合的交集情况
    num = sum(map(lambda x: bool(pwd&x), possible))
    return security.get(num, 'sorry.')

print(checkPwd('abcdefj234,.JE'))
```

4.1.5 字符串应用案例精选

- 补充：使用`itertools.groupby()`函数。

```
from itertools import groupby
```

```
def rules(ch):  
    if '0'<=ch<='9': return 'digits'  
    if 'a'<=ch<='z': return 'lowercase'  
    if 'A'<=ch<='Z': return 'uppercase'  
    if ch in '.,_': return 'punctrations'
```

```
def check(pwd):  
    security = {1:'weak', 2:'below middle', 3:'above middle', 4:'strong'}  
    num = len(tuple(groupby(sorted(pwd), key=rules)))  
    return security.get(num, 'not suitable')
```

```
print(check('acA3Ba4,'))
```

4.1.5 字符串应用案例精选

- **补充案例：** 编写程序，把一个英文句子中的单词倒置，标点符号不倒置，例如 I like beijing. 经过函数后变为： beijing. like I

```
def rev1(s):  
    return ' '.join(reversed(s.split()))  
  
def rev2(s):  
    t = s.split()  
    t.reverse()  
    return ' '.join(t)  
  
def rev3(s):  
    '''字符串整体逆序，分隔，再各单词逆序'''  
    t = ''.join(reversed(s)).split()  
    t = map(lambda x: ''.join(reversed(x)), t)  
    return ' '.join(t)
```

4.1.5 字符串应用案例精选

■ 补充案例： 编写程序，查找一个字符串中最长的数字子串。

```
def longest(s):  
    result = []  
    t = []  
    for ch in s:  
        if '0' <= ch <= '9':  
            t.append(ch) # 遍历字符串中所有字符  
            # 遇到数字，记录到临时变量  
        elif t:  
            result.append(''.join(t)) # 遇到非数字，把临时的连续数字记下来  
            t = []  
    if t:  
        result.append(''.join(t)) # 考虑原字符串以数字结束的情况  
    if result:  
        return max(result, key=len)  
    return 'No'
```

4.1.5 字符串应用案例精选

```
# 选择法
def longest(s):
    length = len(s)
    start = 0
    span = (0, 0)
    for pos in range(length):
        if s[pos].isdigit() and (pos==0 or not s[pos-1].isdigit()):
            start = pos
        elif ((not s[pos].isdigit()) and s[pos-1].isdigit()
              and pos-start>span[1]-span[0]):
            span = (start, pos-1)
    # 字符串以数字结束的情况
    if s[pos].isdigit() and pos-start>=span[1]-span[0]:
        span = (start, pos)
    return s[span[0]:span[1]+1]
```

4.1.5 字符串应用案例精选

```
from collections import Counter

def checkAndModify(word):
    # 待检测单词的字母频次
    fre = dict(Counter(word))
    # 待测单词中各字母频次与所有候选单词的距离，即字母频次之差
    similars = {w:[fre[ch]-words[w].get(ch,0) for ch in word]
                +[words[w][ch]-fre.get(ch,0) for ch in w]
                for w in words}
    # 返回最接近的单词，即字母频次之差的平方和最小的单词
    return min(similars.items(),
               key=lambda item:sum(map(lambda i:i**2, item[1])))[0]
```

4.1.5 字符串应用案例精选

候选单词

```
words = {'good', 'hello', 'world', 'python', 'fuguo',  
         'yantai', 'shandong', 'great'}
```

每个单词中字母频次

```
words = {word:dict(Counter(word)) for word in words}
```

测试

```
for word in ['god', 'hood', 'wello',  
            'helo', 'pychon', 'guguo', 'shangdong']:  
    print(word, ': ', checkAndModify(word))
```

4.1.5 字符串应用案例精选

- **补充案例：**有一些句子和一些关键词，现在想找出包含至少一个关键词的那些句子（文本嗅探），或者想进一步计算每个句子中的关键词占比（句子中所有关键词长度之和/句子长度）。关键词占比是比较常用的一个文本分类标准。

4.1.5 字符串应用案例精选

```
def check(sentences, words):  
    '''返回包含至少一个关键词的句子列表'''  
    return [sentence \  
            for sentence in sentences\  
            if sum(sentence.count(word) for word in words)>0]
```

4.1.5 字符串应用案例精选

```
sentences = ['This is a test.',  
             'Beautiful is better than ugly.',  
             'Explicit is better than implicit.',  
             'Simple is better than complex.',  
             'Sparse is better than dense.',  
             'Readability counts.',  
             'Now is better than never.']  
words = ['test', 'count', 'dense', 'is', 'simple']
```

4.1.5 字符串应用案例精选

```
result = check(sentences, words)
for item in result:
    print(item)

print('='*30)
# 计算每个句子中所有关键字总长度的占比
d = {sentence:round(sum(sentence.count(word)*len(word)\
                        for word in words)/len(sentence),3)\
     for sentence in result}
for item in d.items():
    print(item)
```

4.1.5 字符串应用案例精选

- **补充案例：**给定任意字符串，查找其中每个字符的最后一次出现，并按每个字符最后一次出现的先后顺序依次存入列表。例如对于字符串'**abcda**'的处理结果为['b', 'c', 'd', 'a']，而字符串'**abcbda**'的处理结果为['c', 'b', 'd', 'a']。

4.1.5 字符串应用案例精选

```
s = 'aaaabcdawerasdfasdfwerngsnnvAAAwEB3a'
```

```
# 使用列表
```

```
result = []
```

```
for ch in s:
```

```
    if ch in result:
```

```
        result.remove(ch)
```

```
    result.append(ch)
```

```
print(result)
```

4.1.5 字符串应用案例精选

使用正则表达式

```
import re
```

```
print(re.findall(r'(\w)(?!.*\1)', s))
```

4.1.5 字符串应用案例精选

使用有序字典+内置函数

```
from collections import OrderedDict
```

```
print(list(reversed(OrderedDict.fromkeys(reversed(s)))))
```

4.1.5 字符串应用案例精选

使用推导式

```
result = {ch:s.rindex(ch) for ch in set(s)}  
result = sorted(result.items(), key=lambda item:item[1])  
result = [item[0] for item in result]  
# result = list(map(lambda item:item[0], result))  
print(result)
```

4.1.5 字符串应用案例精选

使用内置函数

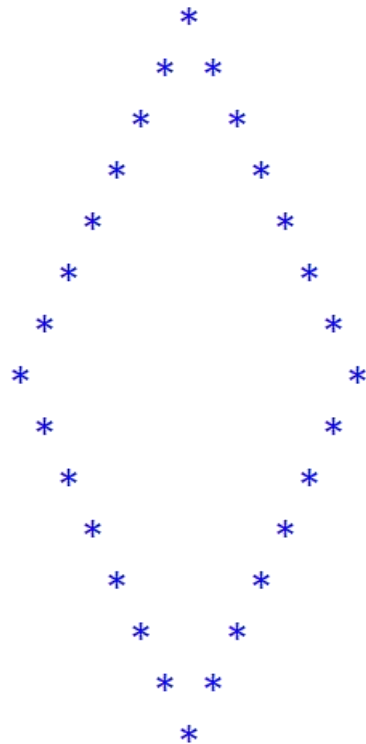
```
print(sorted(set(s), key=lambda ch: s.rindex(ch)))  
print(sorted(set(s), key=s.rindex))
```

4.1.5 字符串应用案例精选

- 补充案例：绘制空心菱形图案

```
def func(n):  
    def process(t):  
        if t.count('*') > 2:  
            start = t.index('*')  
            stop = t.rindex('*')  
            return t[:start+1]+' '*(stop-start-1)+t[stop:]  
        else:  
            return t  
    for i in range(n):  
        t = (' '*i).center(n*2)[1:]  
        print(process(t))  
    for i in range(n, 0, -1):  
        t = (' '*i).center(n*2)[1:]  
        print(process(t))
```

func(8)



4.1.5 字符串应用案例精选

另一种方法

```
def func(n):
```

```
    for i in range(n):
```

```
        print('*'.rjust(n-i)+'*'.rjust(2*i)[:2*i])
```

```
    for i in range(n-2, -1, -1):
```

```
        print('*'.rjust(n-i)+'*'.rjust(2*i)[:2*i])
```

```
func(8)
```

4.2 正则表达式

- 正则表达式使用某种预定义的模式去匹配一类具有共同特征的字符串，主要用于处理字符串，可以快速、准确地完成复杂的查找、替换、删除等处理要求，在文本编辑与处理、网页爬虫之类的场合中有重要应用。
- Python中，re模块提供了正则表达式操作所需要的功能。

4.2.1 正则表达式语法

元字符	含义
.	英文半角圆点字符默认匹配除换行符以外的任意单个字符，使用标志位 <code>re.S</code> 声明为单行模式时也可以匹配换行符。如果要匹配字符串中的圆点字符，需要在前面加反斜线，即 <code>'\.'</code> ；圆点在表示范围的方括号中是普通字符，只匹配圆点本身
*	匹配星号前面字符或子模式的0次或多次重复
+	匹配加号前面字符或子模式的1次或多次重复
-	在方括号中用来表示范围（例如 <code>'[0-9]'</code> 可以匹配任意单个数字字符），在其他位置表示普通减号字符
	匹配位于竖线之前或之后的模式，匹配其中任意一个，可以连用表示多选一
^	1) 匹配以 <code>^</code> 符号后面的字符或模式开头的字符串； 2) 在方括号中开始处表示不匹配方括号里的字符
\$	匹配以 <code>\$</code> 符号前面的字符或模式结束的字符串
?	1) 表示问号之前的字符或子模式可有可无； 2) 当问号紧随 <code>*</code> 、 <code>+</code> 、 <code>?</code> 、 <code>{n}</code> 、 <code>{n,}</code> 、 <code>{,m}</code> 、 <code>{n,m}</code> 这几个元字符后面时，表示匹配模式是“非贪心的”。“非贪心的”模式匹配搜索到的、尽可能短的字符串，而默认的“贪心的”模式匹配搜索到的、尽可能长的字符串。例如， <code>re.findall('abc{,3}?', 'abccc')</code> 返回 <code>['ab']</code> ， <code>re.findall('abc{,3}', 'abccc')</code> 返回 <code>['abccc']</code>

4.2.1 正则表达式语法

<code>\num</code>	1) 使用原始字符串或num前面有两个反斜线时表示前面子模式的编号，num按十进制数理解。例如，<code>r'(.)\1'</code>匹配两个连续的相同字符，<code>\1</code>表示当前正则表达式中编号为1的子模式内容在这里又出现了一次。整个正则表达式编号为0，肉眼可见的第一对圆括号是编号为1的子模式，肉眼可见的第二对圆括号是编号为2的子模式，以此类推。 2) 不使用原始字符串且num前面只有一个反斜线时表示转义字符，num按八进制数理解。例如，转义字符<code>'\101'</code>匹配字符'A'，<code>'\141'</code>匹配'a'，<code>'\60'</code>和<code>'\060'</code>匹配字符'0'。
<code>\f</code>	匹配一个换页符
<code>\n</code>	匹配一个换行符
<code>\r</code>	匹配一个回车符
<code>\b</code>	匹配单词头或单词尾
<code>\B</code>	与 <code>'\b'</code> 含义相反，匹配单词内部
<code>\d</code>	匹配任意单个数字字符， <code>'\d'</code> 等价于 <code>'[0-9]'</code>
<code>\D</code>	与 <code>'\d'</code> 含义相反， <code>'\D'</code> 相当于 <code>'[^0-9]'</code> ，匹配除数字之外的任意单个字符
<code>\s</code>	匹配单个任意空白字符，包括空格、制表符、换页符、换行符， <code>'\s'</code> 等价于 <code>'[\f\n\r\t\v]'</code>
<code>\S</code>	与 <code>'\s'</code> 含义相反，匹配除空白字符之外的任意单个字符
<code>\w</code>	匹配任意单个字母、汉字、数字以及下划线
<code>\W</code>	与 <code>'\w'</code> 含义相反
<code>()</code>	将位于圆括号内的内容作为一个整体来对待，称为一个子模式

4.2.1 正则表达式语法

$\{m,n\}$ $\{m,\}$ $\{,n\}$ $\{m\}$	按大括号中指定的次数进行匹配， $\{m,n\}$ 表示前面的字符或子模式重复m到n次， $\{m,\}$ 表示前面的字符或子模式重复m次至任意多次， $\{,n\}$ 表示前面的字符或子模式重复0~n次， $\{m\}$ 表示前面的字符或子模式恰好出现m次，注意大括号内任何位置都不要有空格。例如 $\{3,8\}$ 表示前面的字符或模式重复3~8次
$[]$	表示范围，匹配位于方括号中的任意单个字符，如果方括号内以^开始则表示不匹配方括号内的字符。例如， $[a-zA-Z0-9]$ 可以匹配单个任意大小写字母或数字， $[\^a-zA-Z0-9]$ 可以匹配除英文字母和数字字符之外的任意单个字符

4.2.1 正则表达式语法

- ✓ 最简单的正则表达式是普通字符串，可以匹配自身
- ✓ `'[pjc]ython'` 可以匹配 `'python'`、`'jython'`、`'cython'`
- ✓ `'[a-zA-Z0-9]'` 可以匹配一个任意大小写字母或数字
- ✓ `'[^abc]'` 可以一个匹配任意除 `'a'`、`'b'`、`'c'` 之外的任意字符
- ✓ `'python|perl'` 或 `'p(ython|erl)'` 都可以匹配 `'python'` 或 `'perl'`
- ✓ 子模式后面加上问号表示可选。`r'(http://)?(www\.)?python\.org'` 只能匹配 `'http://www.python.org'`、`'http://python.org'`、`'www.python.org'` 和 `'python.org'`
- ✓ `'^http'` 只能匹配所有以 `'http'` 开头的字符串
- ✓ `(pattern)*`：允许模式重复0次或多次
- ✓ `(pattern)+`：允许模式重复1次或多次
- ✓ `(pattern){m,n}`：允许模式重复m~n次

4.2.1 正则表达式语法

- ✓ `'(a|b)*c'`: 匹配多个（包含0个）a或b，后面紧跟一个字母c。
- ✓ `'ab{1,}'`: 等价于`'ab+'`，匹配以字母a开头后面带1个至多个字母b的字符串。
- ✓ `'^[a-zA-Z]{1}([a-zA-Z0-9._]){4,19}$'`: 匹配长度为5-20的字符串，必须以字母开头并且可带字母、数字、“_”、“.”的字符串。
- ✓ `'^(\w){6,20}$'`: 匹配长度为6-20的字符串，可以包含字母、数字、下划线。
- ✓ `'^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$'`: 检查给定字符串是否为合法IP地址，圆点前加反斜线是为防止解释为正则表达式元字符匹配任意字符。注意，正则表达式只进行形式检查，并不保证IP地址有效。
- ✓ `r'^(13[0-9]|15[012356789]|17[678]|18[0-9]|14[57])[0-9]{8}$'`: 检查给定字符串是否为手机号码的形式。
- ✓ `'^[a-zA-Z]+$'`: 检查给定字符串是否只包含英文字母大小写。
- ✓ `'^\w+@(\w+\.)+\w+$'`: 检查给定字符串是否为合法电子邮件地址。
- ✓ `r'(\w)(?!.*\1)'`: 查找字符串中每个字符的最后一次出现。
- ✓ `r'(\w)(?=.*\1)'`: 查找字符串中所有出现次数大于1的字符。

4.2.1 正则表达式语法

- ✓ `'^(\-)?\d+(\.\d{1,2})?$'`: 检查给定字符串是否为最多带有2位小数的正数或负数。
- ✓ `'[\u4e00-\u9fa5]'`: 匹配给定字符串中所有汉字。
- ✓ `'^\d{18}|\d{15}$'`: 检查给定字符串是否为合法身份证格式。
- ✓ `'\d{4}-\d{1,2}-\d{1,2}'`: 匹配指定格式的日期, 例如2024-1-31。
- ✓ `'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[,._]).{8,}$'`: 检查给定字符串是否为强密码, 必须同时包含英语字母大写字母、英文小写字母、数字或特殊符号(如英文逗号、英文句号、下划线), 并且长度必须至少8位。
- ✓ `'(?:!.*[\'\"\\/;=%?]).+':` 如果给定字符串中包含'、"、/、;、=、%、?则匹配失败。
- ✓ `'(.+)\1+'`: 匹配任意字符的两次或多次重复出现。
- ✓ `'((?P<f>\b\w+\b)\s+(?P=f))'`: 匹配连续出现两次的单词。
- ✓ `'((?P<f>.)?(?P=f)(?P<g>.)?(?P=g))'`: 匹配AABB形式的成语或字母组合。

4.2.1 正则表达式语法

- ✓ `'\d+(?=[a-z]+)'`: 匹配连续的数字并且最后一个数字跟着小写字母。
- ✓ `'\d+(?![a-z]+)'`: 匹配连续的数字, 并且最后一个数字后面不能跟小写字母。
- ✓ `'(?:<=[a-z])\d+'`: 匹配连续的数字, 并且第一个数字的前面是小写字母。
- ✓ `'(?:<![a-z])\d+'`: 连续的数字, 并且第一个数字的前面不能小写字母。
- ✓ `'\d{3}(?!\d)'`: 匹配三位数字, 而且这三位数字的后面不能是数字。
- ✓ `r'\b(?:!abc)\w+\b'`: 匹配不包含连续字符串**abc**的单词。
- ✓ `'(?:<![a-z])\d{7}'`: 匹配前面不是小写字母的七位数字。
- ✓ `r'((\w)\2(?:!\2)(\w)\3)'`: 匹配**AABB**形式的四字词语。
- ✓ `'(?:<=(\w{4})>)(.*)(?=<\/\1>)'`: 匹配"`<span> hello world </span>`"中的span和hello world。

4.2.6 正则表达式应用案例

■ **补充案例：** 使用正则表达式批量检查网页文件是否被嵌入**iframe**框架。

```
import os
import re

def detectIframe(fn):
    # 存放网页文件内容的列表
    content = []
    with open(fn, encoding='utf8') as fp:
        # 读取文件所有行，删除两侧的空白字符，然后添加到列表中
        for line in fp:
            content.append(line.strip())
    # 把所有内容连接成字符串
    content = ' '.join(content)
    # 正则表达式
    m = re.findall(r'<iframe\s+src=.*?></iframe>', content)
    if m:
        # 返回文件名和被嵌入的框架
        return {fn:m}
    return False
```

4.2.6 正则表达式应用案例

```
# 遍历当前文件夹中所有html和htm文件并检查是否被嵌入框架
for fn in (f for f in os.listdir('.') if f.endswith(('.html', '.htm'))):
    r = detectIframe(fn)
    if not r:
        continue
    # 输出检查结果
    for k, v in r.items():
        print(k)
        for vv in v:
            print('\t', vv)
```